Liquid-Propellant Rocket Engine DAQ Simulation Documentation

LabVIEW–Python ground-test demonstrator

LabVIEW + Python • Software-only demonstrator • October 2026

Executive summary

This project explores how a ground-test Data Acquisition (DAQ) system could monitor a liquid-propellant rocket engine (LPRE) during a static hot-fire test. It uses a Python telemetry source and four LabVIEW Virtual Instruments (VIs) to demonstrate pressure simulation, TCP reception, live monitoring, fault detection, simulated flow control, and initial logging work.

The important boundary: this is not a functioning engine test controller. It has no physical sensors, pumps, valves, propulsion hardware, or qualified safety interlocks. Several assignment-scale capabilities are proposed but not implemented or fully validated.

1. Physical context: what would be monitored

In a pump-fed LPRE, fuel and oxidizer travel from separate tanks through pumps and feed lines to the combustion chamber. Pumps raise propellant pressure to support injection. Combustion produces hot, high-pressure gas that expands through the nozzle to generate thrust. A ground-test DAQ would observe pressures, temperatures, flow rates, vibration, and device states at critical points along that path.

A practical hot-fire monitoring chain is: physical sensor → signal conditioning → DAQ hardware → scaling and timestamps → live operator display → alarms and recorded data. This project simulates only parts of that chain in software.

2. Assignment targets versus demonstration scope

The original assignment notes call for the following target architecture. “Target” does not mean the feature is currently implemented.

Area	Assignment target	Current position
Slow analog	50 channels at 1 kS/s	Not demonstrated in LabVIEW
Fast analog	25 channels at 50 kS/s	Not demonstrated in LabVIEW
Digital I/O	30 inputs and 30 outputs	Not integrated with actual I/O
Analog outputs	2 channels, 0–10 V	No hardware output
Processing	Scaling to engineering units	Single-pressure scaling demonstrated
Control	Two-valve control; pre-check, start, hold, shutdown	Simplified flow/stop logic; no full sequence

Safety	Threshold alarms, latched abort, safe outputs	Simulated threshold and latch tests
Logging	Timestamped TDMS records	Preliminary CSV logging workflow; TDMS not verified
Analysis	Live plots and FFT	Live plots; FFT proposed
Communication	TCP synthetic telemetry	Single-pressure TCP reception demonstrated

3. System architecture

The software components are deliberately modular. They can be developed and demonstrated independently; a fully integrated master application is not yet established.

Component	Purpose	Observed or reported behavior
Python telemetry server	Provide synthetic measurements over TCP	Pressure stream received in LabVIEW; notes also report Python-side multichannel generation and unit tests
Simulated_Pressure.vi	Create pressure values, inject a fault, display alarm	Normal 2.0–2.2 bar; injected 3.5 bar; overpressure comparison at 3.0 bar; abort latch tested
TCP_Acquisition.vi	Read pressure data from a TCP server	Port 5000 connection, continuous pressure reception, and disconnection response tested
Flow_Control.vi	Model a valve opening command and stop latch	80% opening gives 40 L/min in simplified model; emergency stop forces 0; reset restores flow
DAQ_Main.vi	Provide master front panel and first-pass logging	Standalone pressure indicator/chart and trial alarm documented; full orchestration/logging not verified

4. Data path: from simulation to LabVIEW

The Python server produces synthetic readings and sends them over a local TCP connection. The LabVIEW TCP receiver connects to port 5000, reads the incoming pressure value, and displays it. During the communication fault test, closing the server led to TCP Error 66 and stopped the receiver loop. TCP Close Connection was wired, but its execution was not separately confirmed.

This is a useful acquisition prototype, but receiving one formatted pressure field is not the same as decoding and time-aligning 75 analog channels plus digital inputs at the assignment's requested sample rates.

5. Pressure simulation and safety demonstration

Simulated_Pressure.vi generates a normal pressure signal using:

P (bar) = 2.0 + 0.2 × Random(0, 1)

This produces values of roughly 2.0–2.2 bar. An injection switch substitutes 3.5 bar so the overpressure comparator (> 3.0 bar) can be tested. The overpressure indicator clears when the input returns to normal, but the separate abort latch remains on until the VI is restarted in the documented test.

Test	Expected behavior	Recorded result
Normal pressure	No overpressure or abort	Passed
Inject 3.5 bar	Overpressure and abort on	Passed
Restore normal input	Overpressure off; abort remains latched	Passed
Restart VI	Abort initializes false	Passed

All pressure values and limits here are demonstration settings, not actual LPRE operating limits. A production design would require validated sensor ranges, independent safety logic, explicitly defined reset authority, and fault-tolerant hardware outputs.

6. Flow-control simulation

Flow_Control.vi models a simplified relationship between a commanded valve-opening percentage and a synthetic flow-rate indicator. At 80% opening, the expected displayed flow is 40 L/min. Pressing Emergency Stop forces indicated flow to zero; releasing it does not restore flow until a reset is applied. An active emergency-stop input takes precedence over reset.

Action	Expected indicated flow	Recorded result
Opening set to 80%	40 L/min	Passed
Emergency stop pressed	0 L/min	Passed
Emergency stop released	Still 0 L/min	Passed
Reset pressed after release	40 L/min	Passed
Stop simulation	Loop ends	Passed

The model does not compute physical valve characteristics, pump behavior, pressure losses, or fluid dynamics. It is a software exercise in control-state handling, not closed-loop propellant-flow regulation.

7. Main VI and data recording

DAQ_Main.vi was intended as the primary monitoring and logging interface. The notes document an internal synthetic pressure source, numeric pressure display, waveform chart, and a trial threshold indicator at 2.15 bar. That threshold was used only to exercise alarm behavior. The intended logging scheme creates run-specific CSV files with date/time-based filenames, rather than overwriting one shared file.

A complete, validated run-specific CSV writer, a TDMS acquisition record, coordinated execution of the other three VIs as subVIs, and read-back verification are not established by the supplied notes. They should remain described as preliminary or future work.

8. Validation summary and limitations

What the notes support: standalone pressure generation and fault injection; pressure alarm/abort latch behavior; basic TCP reception and disconnect detection; simplified valve/flow emergency-stop behavior. The notes also report 18/18 Python unit tests and a 12-frame live TCP test, although the underlying test report is not included in this document.

What remains unproven: physical DAQ channels; 50 kS/s acquisition; synchronized multichannel scaling and buffering; integrated state sequencing; dual-valve hardware logic; TDMS logging; FFT vibration analysis; qualified abort/safe outputs; and complete end-to-end integration.

9. Suggested next engineering steps

- Define a message schema with channel IDs, units, timestamps, sequence numbers, and error cases.
- Test sustained throughput, dropped samples, reconnect behavior, and buffering separately for slow and high-rate channels.
- Turn standalone LabVIEW loops into callable subVIs with explicit error and state interfaces.
- Implement and verify a full pre-check → start → hold → shutdown/abort state machine.
- Record timestamped data with metadata; verify CSV or TDMS files by reading them back.
- Add synthetic vibration data and FFT tests with known frequency components, then document measured results.

10. Conclusion

The LabVIEW–Python prototype demonstrates a clear starting point for an LPRE hot-fire monitoring workflow: generate simulated pressure, receive it over TCP, visualize the signal, detect faults, and exercise simulated stop-state logic. Its value is as a modular software demonstration and a basis for discussing engineering trade-offs—not as a completed high-channel-count DAQ or engine safety system.

Repository: https://github.com/aniruddhac99/liquid-rocket-hotfire-daq-sim

Source note: This walkthrough was edited from the uploaded 22-page working document. It consolidates repeated instructions and preserves test observations as reported, without claiming independent verification of the LabVIEW files or actual DAQ hardware.